CIS 4910

Computer Science & Engineering Project: Capstone

WEEK 2 • FALL 2026

Professor Scot Hollingsworth

scoth@usf.edu • Office Hours by appointment (Microsoft Teams)

Bellini College of Artificial Intelligence, Cybersecurity and Computing

Status Report Issues (2026)

1. Use the Template (Status reports and Timesheets) Should be 2 files, {Word, pdf, text, etc.}, excel sheet (No pdf or picts of the sheet)
2. One or more feel behind already due to a lack of tech experience.
3. Please try and refrain from AI on the status reports, they are not that hard and easy to tell, plus the few who used them followed the rules and said so. (you won't lose points but give it a try!)
4. Remember NO LATE WORK, only 1 submission! (Please don't ask)
5. I primary communicate using the course announcements.
6. Everyone is welcome to open discussions.

Side Bar !

Trust Vs Performance

https://www.youtube.com/watch?v=PTo9e3ILmms

Agenda for Today

1. Role call: first contact with your sponsor
2. Expectations for Week 3 Presentations
3. ABET Engineering Standards
4. Development methodologies
5. Waterfall Vs Agile (Comparisons)
6. AI As an Accelerant
7. Project flow: Epics → Features → Stories & Specifications
8. Requirements as User Stories
9. Specific Examples
10. Tasks for the week

BY TEAM

Role Call: Two Questions

Has your first meeting occurred?

Confirm whether your team has met with your sponsor yet.

Have you made first contact?

Copy Professor Hollingsworth on all emails to your sponsor going forward.

Due next week (Week 3): your team status report & Video (Wed) and a short (10min<) presentation to the class (Friday). No points, No Grade, a chance to get comfortable with presentations. Use the template in Canvas. Share the business problem (EPIC) and first cut at a few requirements. Opportunity for a stress-free day to practice an in-class presentation.

BY TEAM

Week 3: Project Presentation

I Look for 2 specific items (10 minutes max): No Grade, EXPOSURE !

Your Presence

- Are all members present
- Introductions
- Speech
- Volume
- Stance
- Content and delivery
- What to do when others speak (hands, stance, eyes)

Your Project
USE TEMPLATE

- Name of project
- Sponsor
- What type of technology
- Basic Requirements
- How will the team approach the problem
- What are some major constraints you will encounter

BY TEAM

Presentation Skills & Suggestions

1. Start with Audience, not the sides. What do you want your audience to know / feel?
2. Clarify your key message. Start with “WHY”, not the what and definitely not the “how”
3. How do you plan to “solve the business problem”: High level tech
4. Use a “Killer” opening line or story to grab the audience attention. End with a major point.
5. Make each slide convey a single thought, do not make slides with too much “Clutter”. (My Slides are not the greatest examples)
6. Practice your team’s delivery. Record yourselves and watch the recording. Be aware of where you look when you present and when others on the team present. Yes, video in class is allowed for this purpose only!
7. Don’t just “inform” your audience, inspire them to both understand and want more details about your topic.

What is required in a single "Credit Hour"

Federal Definition of Credit Hour: a credit hour is an amount of work represented in intended learning outcomes and verified by evidence of student achievement that is an institutionally established equivalency that reasonably approximates

1. Not less than one hour of classroom or direct faculty instruction and a minimum of two hours out of class student work each week for approximately fifteen weeks for one semester or trimester hour of credit, or ten to twelve weeks for one quarter hour of credit, or the equivalent amount of work over a different amount of time, or
2. At least an equivalent amount of work as required outlined in item 1 above for other academic activities as established by the institution including laboratory work, internships, practica, studio work, and other academic work leading to the award of credit hours.
3. The credit hour definition is a ***minimum standard*** that does not restrict an institution or course from setting a higher standard that requires more student work per credit hour.
4. The definition does not dictate particular amounts of classroom time versus out-of-class student work.
5. Expected time for this 3-credit hour class based on the definition of 1 credit hour above:
    a) Facility Instruction for 15 weeks X 3 CH = 45 hours of instruction. My portion of this contact for the course will be weekly face-to-face or Online Lectures and participation with discussions.
    b) Students work for 2 hours per week X 15 weeks X 3 CH = 90 hours of student work.
    c) Total hours 60 + 90 = 150 hours / 15 Weeks for this section, equates to **10 hours per week at a minimum.**
    d) A course project of the following should yield the following hours:
6. Project Teams of 3: 10 hours per week per team member = **450-hour** project.
7. Project Teams of 4: 10 hours per week per team member = **600-hour** project.
8. Timesheets are used to assist the instructor in ensuring project teams are on track with their project. Students should be as accurate as possible when completing timesheets; these documents are required as part of the course and must never be falsified.

See the university accreditation definition below for credit hours as defined by the *Southern Association of Colleges and Schools, Commission on Colleges, 1866 Southern Lane, Decatur, Georgia 30033-4097:*

BORING, BUT IMPORTANT

ABET & Engineering Standards

ABET is the accrediting body behind your degree, and this course exists in part to satisfy its capstone design requirements.

A handful of definitions here matter for how this class, and your career, are structured.

ACCREDITATION

What Is ABET?

ABET is a nonprofit, non-governmental organization that accredits college and university programs in engineering, computing, engineering technology, and applied and natural science.

- USF's BS in Computer Engineering is accredited by ABET's Engineering Accreditation Commission (EAC)
- USF's BS in Computer Science is accredited by ABET's Computing Accreditation Commission (CAC)
- More at www.abet.org

Critical Thinking

Capstone Design: Why This Course Exists

- This course follows ABET's EAC definitions, curriculum, and program criteria for capstone design
- Graduating from an ABET-accredited program is the first step to becoming an engineer

ABET's definition

Capstone design is "a culminating major engineering design experience that (1) incorporates appropriate engineering standards and multiple constraints, and (2) is based on the knowledge and skills acquired in earlier course work."

Source: ABET EAC Criteria document

Critical Thinking

Complex Problems & Team

Complex Engineering Problems

Involve wide-ranging or conflicting technical issues, have no obvious solution, aren't covered by current standards/codes, involve diverse stakeholders or multiple disciplines, or carry significant consequences.

Team

More than one person working toward a common goal. Should include individuals of diverse backgrounds, skills, or perspectives.

Critical Thinking

WHAT YOU'RE ACTUALLY GRADED ON

Student Outcomes

> *"Student outcomes describe what students are expected to know and be able to do by the time of graduation: the knowledge, skills, and behaviors students acquire as they progress through the program."*
>
> Source: ABET EAC Criteria

- In short: the stuff we teach you, and you need to learn
- Look for ABET Student Outcomes on your course syllabi, sometimes "disguised" in other words, but they're there

Critical Thinking

OUR STUDENT OUTCOMES

What You'll Be Able to Do

- Analyze a complex computing problem and apply principles of computing and other disciplines to identify solutions
- Design, implement, and evaluate a computing-based solution to meet a given set of requirements
- Communicate effectively in a variety of professional contexts
- Recognize professional and ethical responsibilities and make informed judgments in computing practice
- Function effectively as a member or leader of a team
- Apply computer science theory and software development fundamentals to produce computing-based solutions

Critical Thinking

PUTTING IT TOGETHER

This Course, In One Slide

This course teaches Engineering Design applied to Complex Engineering Problems, where students work in teams to tackle real-world industry problems.

- Key Student Outcomes are covered and assessed throughout
- You will learn and follow a real product development process

Critical Thinking

TWO MAJOR APPROACHES

Development Methodologies

There are two major approaches to running an engineering project: Waterfall and Agile, plus Lean as a related philosophy.

Today we'll compare how they handle requirements, design, implementation, and testing, and where AI tools change the picture.

THE BIG PICTURE

Waterfall vs. Agile vs. Lean

Waterfall

Sequential phases. Requirements and design happen once, up front, then flow downhill through implementation and test.

Agile

Iterative cycles. Requirements, design, implementation, and test repeat in short increments, in constant contact with the customer.

Lean

A philosophy more than a method. Minimize waste, maximize value, often paired with Agile or Kanban-style flow.

SAME STEPS, DIFFERENT OWNERS

Methodology Steps: Who Does What

Step	Option 1	Option 2
Requirements	Marketing (not developers)	Developers, directly with the customer
Design	Senior engineers only (one “genius” may do it)	Team effort, by the developers
Implementation	Offshore, low-cost labor	Local developers, directly with the customer
Testing	Junior engineers and interns	The developers, directly with the customer

WHERE THE TIME GOES

Waterfall vs. Agile: Time Per Step

Waterfall: front-load it

Most time goes into requirements and design. Fixing flaws later is far more costly. Customer signs off on requirements; developer signs off on delivering them, on schedule.

Agile: iterate continuously

Requirements, design, implementation, and test repeat in cycles, in constant communication with the customer rather than a single up-front phase.

A COMMON QUESTION

Can You Change the Order of the Steps?

Maybe…… A few variations DO exist:



Test-Driven Design (TDD)

Write the test plan before the design, then design to it. High cost to the sprint; some DoD projects use it.



Domain-Driven Design

Organizes around the business domain. Can force staff into detailed silos which are hard to break!



Feature-Driven Design

Originally for very large teams; now common in small teams with short timelines to production (a Lean/Agile variant). Becoming more popular with AI Teams.

This will most likely be the “flavor” of your Agile development cycles for this class.

How Many Times Do You Run the Process?



Once: Do It Right

Waterfall, Gate-Driven, Cleanroom, and Lean's early "Rapid Prototyping" (a gateway to Agile).



Iterate until done

Work in ~two-week increments *(**one week for this class).*** Each increment ships a working product with more features. Stakeholders attend every sprint review.

I expect this detail in status reports / Videos (Show me working features)

OLD SCHOOL MEETS NEW SCHOOL

Waterfall and Agile, Honestly

Waterfall

“Old school,” but it put a man on the moon.” More exacting (Cleanroom-Waterfall). Still used in Manned space flight, and most DoD projects.

Agile

Trendy and in vogue, (Many Flavors) started in software and spread everywhere. Now beginning to blend with other methods. SpaceX is using this model for Star Ship Development.

Reality isn't either/or: many companies do some of both. A hybrid is often better than either extreme; a good question for our company speakers.

WHERE AGILE CAME FROM

Agile History

The Agile Manifesto dates to 2001, written by Kent Beck, Ward Cunningham, Dave Thomas, Jeff Sutherland, Ken Schwaber, Jim Highsmith, Alistair Cockburn, and Bob Martin.

“The ability to create and respond to change in order to succeed in an uncertain and turbulent environment.”

Source: agilealliance.org

“Solutions evolve through collaboration between self-organizing, cross-functional teams utilizing the appropriate practices for their context.”

Source: agilealliance.org

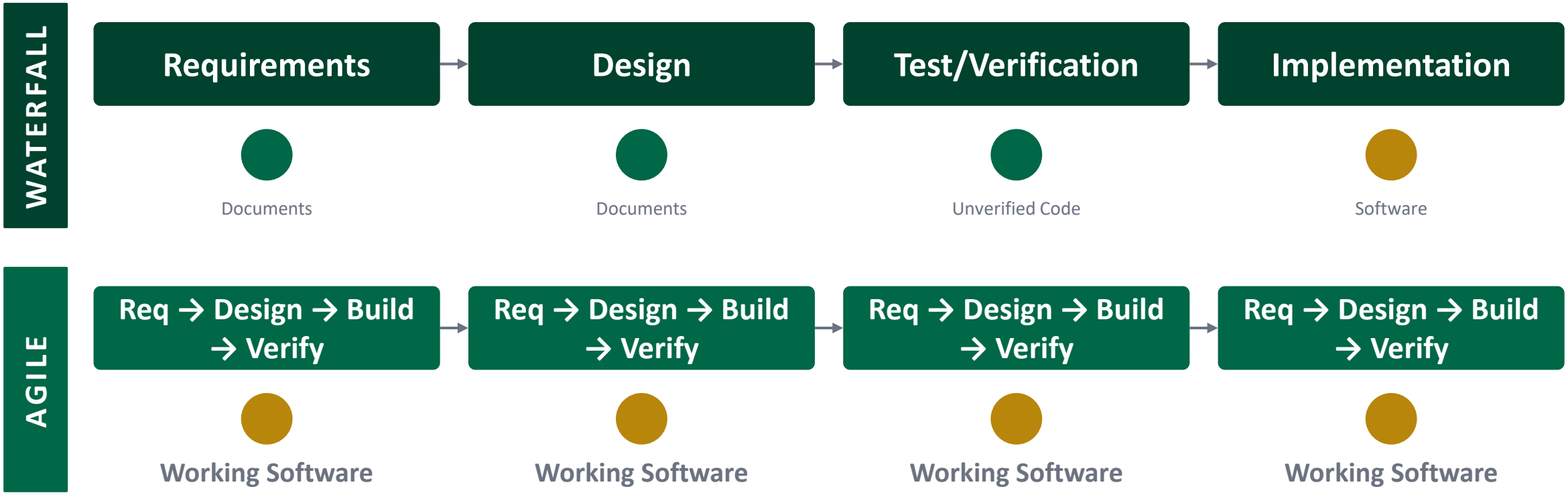
FROM THE MANIFESTO

Agile Development Principles

- Customer satisfaction through early, continuous delivery
- Welcome changing requirements, even late in development
- Deliver working software frequently (Days, Weeks, not Months)
- Daily cooperation between business and developers
- Trust motivated individuals to get the job done
- Face-to-face conversation is the best communication
- Working software is the primary measure of progress
- Maintain a sustainable, constant pace
- Simplicity: maximize the work not done
- Teams regularly reflect and adjust how they work

SAME STEPS, DIFFERENT RHYTHM

Comparing Waterfall to Agile



Source: Scaled Agile Inc.

The big idea: lots of small increments, starting from a Minimum Viable Product, with constant conversation with the customer.

A NEWER APPROACH that is AI Centric

AI-Augmented Software Engineering (AI-SE)

Alongside Waterfall, Agile, and Lean, a fourth approach has emerged in the last few years: not a replacement, but a layer that changes how the existing steps get done.

What it is

Using AI tools throughout the lifecycle: generating code, drafting tests, summarizing requirements, even proposing designs, with a human reviewing every step.

What changes

Speed of the routine work goes up sharply. The engineer's job shifts toward reviewing, verifying, and directing rather than typing every line.

What doesn't change & Enhances

Domain Knowledge, Requirements, Specifications, design judgment, and accountability still belong to the team. AI assistance must be disclosed, same as Week 1's AI policy.

AI-SE IN PRACTICE

One Sprint, with AI in the Loop (Sample)

A realistic walkthrough of where AI tools plug into a single two-week sprint. Every AI-touched step still has a human review step right after it.

1. **AI Draft the story**
   Developer gives the AI tool a rough ask (“customers need to reset their password”). AI drafts a user story with acceptance criteria.
2. **HUMAN Review and refine**
   Developer edits the draft for accuracy, checks it against real customer language, and signs off before it enters the backlog.
3. **AI Scaffold the implementation**
   AI generates a first-pass implementation of code and a drafts a set of unit tests from the approved story.
4. **HUMAN Review, test, and verify**
   Developer reviews the generated code, runs and extends the tests, and verifies the result actually meets the acceptance criteria.
5. **AI Create Automated Testing Harness**
   AI Develops a complete end-2-end testing process that executes for each run of the CI/CD pipeline with human oversight.
6. **HUMAN Disclose and ship**
   AI involvement is logged in the story and the status report, same as Week 1’s AI Usage Policy. The increment ships.

DESIGNING AT A HIGHER LEVEL

Model-Driven Engineering (MDE) & System Modeling

The core idea

Build formal models of the system: structure, behavior, data flow, and treat those models as the primary design artifact, not an afterthought to the code.

Why it matters now

AI tools are good at generating code from a clear model, and good at generating a draft model from a clear requirement, making MDE more practical than it used to be.

In this class: you're *not* required to build formal UML/SysML models, but a simple 1-page diagram of your system's structure or data flow (hand-drawn or AI-assisted) belongs in your Architectural Design Document (DDD); often referred to as an HLA (High Level Architecture).

Break

WHERE AI FITS HERE

AI as an Accelerant, Not a Shortcut

AI tools can speed up implementation dramatically: scaffolding code, drafting test cases, even drafting a first-pass user story from a rough idea.

But they don't get to skip requirements, design, or judgment about whether the result is right. Whichever methodology your team uses, AI assistance on a story, an estimate, or a draft spec must be disclosed. Same rule as Week 1's AI Usage Policy, applied here to Agile artifacts and status reports.

WHAT WE'LL ACTUALLY DO

Our Hybrid Approach in This Class

There's no single best answer to Agile. Focus on the manifesto and its principles. ABET requires documents demonstrating a verifiable process, so this class blends both worlds.

Nail down requirements (They will be refined over 15 weeks)

Best possible requirements and specifications up front, with details, while expecting some change.

Design, then iterate (Starts week 1)

Do the best possible design, then iterate and add functionality by priority until the schedule ends.

Demo and grade (A presentation / review about every 3-5 weeks)

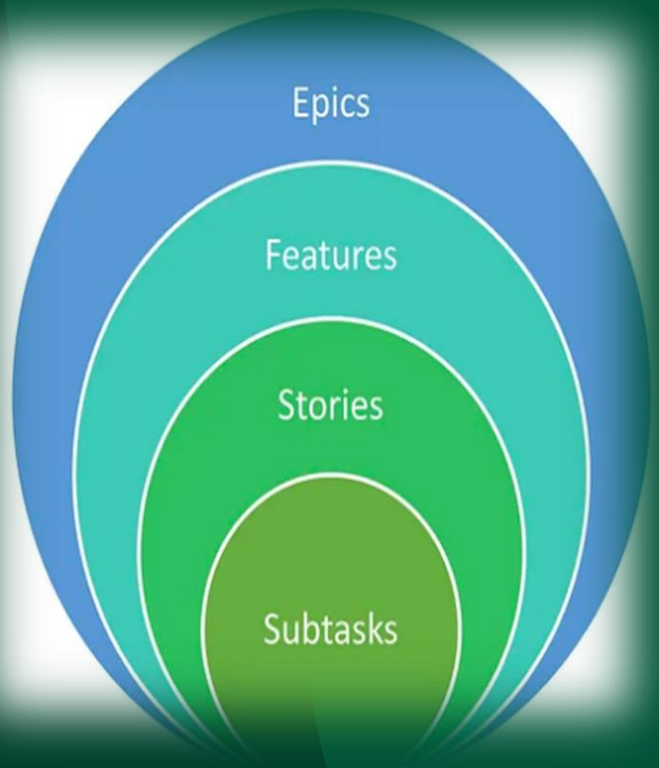
Incremental demos plus a final demo, each tied to a grade. Goal: meet requirements, make the customer happy, document as required, present your progress both informally and formally.

FROM PROBLEM TO PLAN

Epics, Features, Stories, and Specs

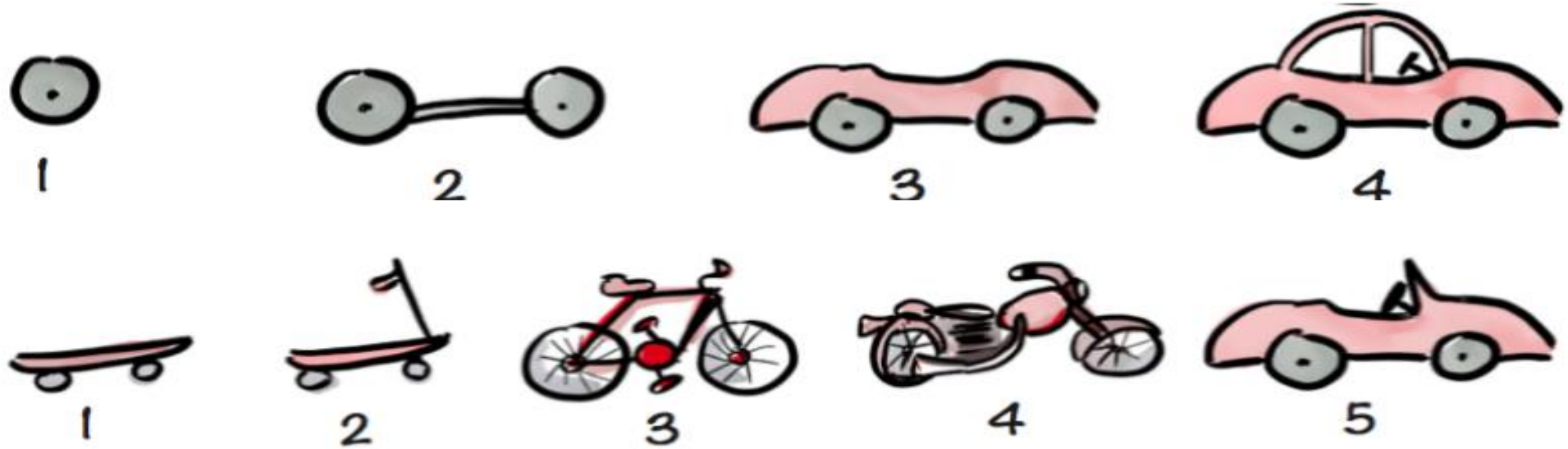
Every project starts with a business problem: an Epic. From there it breaks down into Features, then into Stories small enough to build in under a week. We will not deal with Subtasks in this course.

We'll also draw a hard line between a Requirement (the WHAT) and a Specification (the HOW).



Views of the progression of a project

Creation of a Car by Agile Increments



As a Driver, I want a car that seats up to 4 people, **so that I can travel up to 200 miles without refueling**.

Acceptance criterion: A car that can fit four adults comfortably that can travel 200 miles on a single tank of gas.

BREAKING DOWN THE PROBLEM

Epics → Features → Stories

Epic (1 for this course)

A major business problem or user need, often cross-cutting several functions.

→

Features (multiples)

A specific product capability that achieves part of the epic. (Takes more than 1 week)

→

Story (1/pers/week)

A concise piece of functionality from the user's perspective, completable in under a week.

WORKED EXAMPLE: THE RIVER CROSSING

EPIC

How can I move 100 people/day, carrying up to 100 lbs, across this river at under 1-in-1,000,000 risk of falling in?

→

FEATURE

A pedestrian crossing capable of 100 person-trips per day in all weather conditions.

→

STORY

A single-span footbridge with a load-rated, non-slip walking surface and guard rails on both sides.

Requirement: As a city engineer, **I want** a load-rated footbridge with non-slip decking and rails, **so that** 100 people a day can cross safely in any weather.

Acceptance criterion: bridge passes load testing at 100 simultaneous 100 lb crossings with zero structural deflection beyond code.

The “Cutting Process”

Epics → Features → Stories Flow Process

Cutting Process From Epic to Feature

The transformation of an epic into features and subsequently into a user stories is fundamental to agile. This method breaks down objects into specifics task, facilitating planning, execution and follow-up.

Understanding Cutting

- **Epic:** Major object or user need, often cross-cutting several functions
- **Feature:** Specific product capability or enhancement, achieving part of the epic.
- **User Story:** Concise descript of a specific functionality from a users preceptive.

Cutting techniques

Identify the epics objective: what is the user’s need?

- **Breakdown into features:** divide the epic into multiple features.
- **Subdivision into user stories:** Break down each feature into small work segments which can be completed in less than 1 week (This class). Might take 1-4 team members or some subset.



TYPES OF REQUIREMENTS

1. Business Requirements: “Reduce customer checkout time by 20% to 3sec or less”
2. Stakeholder Requirements: “SVP needs to see a dashboard of sales numbers per quarter”
3. **Solution Type Requirements**
   - A. Functional Requirements: Something the system will do; Send an email notification when..
   - B. Non-Functional Requirements: security, storage, reliability, legal, compliance, etc..
   - C. Performance Requirements: Shall load data within 10min, etc..
   - D. Testing Requirements: Must be able to test the features of section A in automated manner

Requirements as User Stories

As a (role), I want (something), so that (benefit)

- Requirements are about the problem space, not the solution space. Use the customer's language as much as possible.
- Every requirement (story) must be: Numbered, Measurable, Unique, and in Priority order.
- Acceptance criteria define when a story is “done” and can be a starting point for test cases.

A CRITICAL DISTINCTION

Requirements vs. Specifications

Requirement (the WHAT)

"Create storage of customer records, indexed by last name." Tells the builder what is being requested.

This is expected by week 3 presentation

Specification (the HOW)

"Create an Oracle database in 3rd normal form, with customer records indexed by last name and zip code." Tells the builder how a requirement is implemented.

Wk5: Expected with some detail
Wk8: Well Refined
Wk12: At the 90% level
Wk15: 100% accurate

Mapping rule: a specification must match its requirement exactly, and carries the same four traits (numbered, measurable, unique, in priority order) plus timeline, dependencies, and constraints. While the original customer requirement might rarely change, the details of the specification change as the project is better understood. The spec will be light in Week 3, but should be solid by week 8.

PULLING IT ALL TOGETHER

SRS Document Layout (USE THE TEMPLATE !!)

A Software Requirements Specification includes:

- Cover page, history page, table of contents
- Introduction: problem statement / Epic, one sentence stating the business problem.
- Glossary, Features (2–5), Requirements as stories (10–20)
- Needs and factors, assumptions, constraints
- Specifications, project plan, change log, testing plan, time log, references



AI & Disclosure, Reminder

- Track every requirements change: Change Records, Change Activities, new stories, removed stories
- If AI helped draft a story, a spec, or a section of your SRS, disclose it. Same as code
- Tracking starts once requirements/specs are accepted, signed off, or graded. This happens in **WEEK #5**

Quantify your Work Process

AI Can only assist in developing stories at a Amazing level, if your input is detailed and accurate.

EXAMPLE ONLY!

Time Tracking System - Epic

The user requires our team to develop a complete, web-based time tracking system that enables an organization to define and manage projects, assign people to projects, capture individual time in 15-minute increments, and produce accurate project and person-based reporting for project management, resource utilization, cost tracking, and accountability.

Major Features

1. *Project & People Management:* create projects, maintain supporting details, and manage assignments.
2. *Time Entry & Tracking:* record individual time in 15-minute increments and calculate totals/cost.
3. *Reporting & Analysis:* provide project-based and person-based reporting for management.

Security Requirements

1. Administrator / Project Manager: manages projects, people, assignments, and project information.
2. Team Member / Employee: records and maintains time against assigned projects.
3. Report Viewer / Management: reviews project and individual time/cost reports.

EXAMPLE ONLY!

Project Management and Time Tracking – Features & Stories

1. Project & People Management

Establish the projects, people, and relationships required for time tracking.

Story 1.1 — Create Project

As a Project Manager, I want to create a project and enter its supporting details, so that the organization has a centralized project record.

Project name, description, client/customer, manager, dates, status, team, budgeted hours, and project cost/budget.

Story 1.2 — Assign People

As a Project Manager, I want to assign people to a project, so that team members can record time against authorized projects.

2. Time Entry & Tracking

Capture accurate individual time against assigned projects.

Story 2.1 — Enter Time

As a Team Member, I want to enter time worked on an assigned project in 15-minute increments, so that my work is accurately recorded.

Users enter 0.25-hour increments with date and optional activity/description.

Story 2.2 — Review Time

Users can review and correct previously entered time, subject to system rules.

Story 2.3 — Calculate Project Cost

As a Project Manager, I want the system to calculate cost from recorded hours and applicable hourly rate.

3. Reporting & Analysis

Turn captured time into useful management information.

Story 3.1 — Project-Based Report

As a Project Manager, I want to generate a report by project, so that I can understand time and cost spent on each project.

Include project, status, team, total hours, hours by person, project cost, budget, and variance.

Story 3.2 — Person-Based Report

As a Manager, I want a report by person, so that I can understand how an individual's time is allocated across projects.

Include person, projects, hours, total hours, cost, and date/time period.

Story 3.3 — Filter Reports

As a Report Viewer, I want to filter by project, person, status, and date range.

EXAMPLE ONLY!

Project Management and Time Tracking – Specification Examples

Story 2.1 — Enter Time

As a Team Member, I want to enter time worked on an assigned project in 15-minute increments, so that my work is accurately recorded.

Specification 2.1.1

Select Assigned Project and Date

As a team member, I want to select an assigned project and the date I worked on it, so that my time is recorded against the correct project and work date.

Specification 2.1.1

Select Assigned Project and Date

As a team member, I want to select an assigned project and the date I worked on it, so that my time is recorded against the correct project and work date.

Specification 2.1.1

Select Assigned Project and Date

As a team member, I want to select an assigned project and the date I worked on it, so that my time is recorded against the correct project and work date.

EXAMPLE ONLY!

Assumptions, Constraints & Test Cases (Sample)

Assumptions

a) **Users are assigned to projects** before they can enter time against those projects.
b) **Time is recorded in hours using 15-minute increments**, with 0.25 hours representing 15 minutes.

Constraints

a) **Time entries must be entered in 15-minute increments**; values such as 0.10 or 0.30 hours are not permitted.
b) **Team members can only enter time for projects to which they are assigned.**

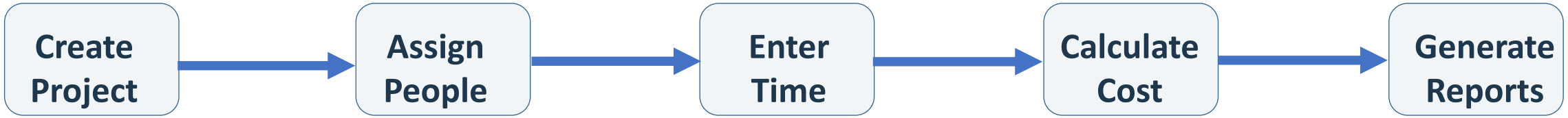
Test Cases

Test Case	Test Description	Expected Result
TC-2.1.2-01	Enter 0.25 hours for an assigned project.	System accepts and saves the 0.25-hour time entry.
TC-2.1.2-02	Enter 1.50 hours for an assigned project.	System accepts and saves the 1.50-hour time entry.
TC-2.1.2-03	Enter 0.30 hours for an assigned project.	System rejects the entry and informs the user that time must be entered in 15-minute (0.25-hour) increments.

EXAMPLE ONLY!

Definition of Done

EXPECTED END-TO-END WORKFLOW



Project / Epic SUCCESS CRITERIA

- Demonstrate a working end-to-end system, not simply individual screens or database components.
- Data entered into projects, people, and time tracking must flow correctly through the application.
- Project and person reports must accurately reflect recorded hours and associated costs.
- The team should be able to demonstrate the complete workflow during the final project presentation.
- Supporting technical work (authentication, Git integration, attachments, deployment, etc.) should support the core business features rather than expand the business scope.

BEFORE YOU LEAVE TODAY

Tasks for the Week

1. Work on requirements and your SRS document.
2. Start proto-typing your project
3. Prepare a short review of your project objective (EPIC Statement) and requirements for Week #3, Every team member presents. Use the template on Canvas. (10min total to include Q&A)

Getting requirements right now is critical. A short-detailed conversation today can save everyone hours later.